[Dung Vu] IPC: Pipes, FIFOs, Message Queues, Shared Memory, Signals

- **Section**: Linux System Programming
- **Topic**: IPC: Pipes, FIFOs, Message Queues, Shared Memory, Signals

---

1. Interprocess Communication Overview

Interprocess Communication (IPC) allows a process to communicate with another process.

UNIX IPC facilities are divided into thread broad functional categories:

- *Communication*: exchange data between processes
- *Synchronization*: synchronize the actions of processes or threads.
- *Signal*: software interrupts, can be used as a synchronization technique.

ipc.png

---

2. Pipes

2.1. Overview

A pipe is an unidirectional data channel.

Reading behavior:

- `read()` from an empty pipe blocks, until one byte is written.
- Once write end is closed and all buffered data has been read, `read()` returns 0.

`PIPE_BUF` **- atomic threshold**:

- Writes ≤ `PIPE_BUF` bytes are atomic guaranteed - multiple writers won't have their data interleaved.
- Writes > `PIPE_BUF`: kernel may split into smaller chunks interleaved with other writers' data.
- Value: 512 (FreeBSD 6.0), 4096 (Tru64 5.1, **Linux**), 5120 (Solaris 8).

Pipe capacity:

- A pipe is a kernel buffer with finite capacity; once full, writers block until a reader consumes data.
- *Linux kernels before 2.6.11*: pipe capacity = page size (e.g., 4096 bytes on x86-32)
  Since Linux kernels 2.6.11: pipe capacity = 65,536 bytes.

2.2. Creating and Using Pipes

```
#include <unistd.h>


int pipe(int filedes[2]);
/* Returns 0 on success, or –1 on error */
```

The situation after a pipe has been created by `pipe()`:

pipe.png

During a `fork()`, the child process inherits copies of its parent’s file descriptors.

pipe2.png

After that:

- The sender should closes the read end of the pipe.
- The receiver should closes the write end of the pipe.

Example 1: Normal tranfer through pipe

```
#include <stdio.h>
#include <unistd.h>


int main()
```

```
{
    int filedes[2];

    if (pipe(filedes) == -1)
    {
        perror("pipe");
        return -1;
    }

    switch (fork())
    {
        case -1:
            perror("fork");
            return -1;

        case 0: /* Child process */
            close(filedes[1]); /* Close unused write end */

            /* Child now reads from pipe */
            char buffer[100];
            while(read(filedes[0], buffer, sizeof(buffer)) > 0)
            {
                printf("Received from parent: %s\n", buffer);
            }

            close(filedes[0]); /* Close read end after writing */
            break;

        default: /* Parent process */
            close(filedes[0]); /* Close unused read end */

            /* Parent now writes to pipe */
            write(filedes[1], "Hello, child!", 14);

            close(filedes[1]); /* Close write end after reading */
            break;
    }

    return 0;
}
```

Output:

```
Received from parent: Hello, child!
```

Example 2: Self-deadlock since a process doesn't closing one end.

Same code on the example above, but we doesn't close the write end in the child process.

```
case 0: /* Child process */
    /* close(filedes[1]); */ /* Write end now isn't closed*/

    /* Child now reads from pipe */
    char buffer[100];
    while(read(filedes[0], buffer, sizeof(buffer)) > 0)
    {
        printf("Received from parent: %s\n", buffer);
    }

    close(filedes[0]); /* Close read end after writing */
    break;
```

Output:

```
Received from parent: Hello, child!
```

We get the same output, main process ends successfully.

However, the child process is still running since it gets a self-deadlock. The pipe still has the write end in the child process, and `read()` blocks it from runnning.

```
$ ps aux | grep pipe_2
dungvd     44574  0.0  0.0   2772  1008 pts/2    S    04:01   0:00 ./pipe_2
```

2.3. popen()/pclose()

```
FILE *popen(const char *command, const char *mode);
/* Returns file stream, or NULL on error */

int pclose(FILE *stream);
```

```
/* Returns termination status of child process, or –1 on error */
```

The `popen()` function:

- Create a pipe with `mode` (`r` - read from callee, `w` - write to callee).
- `fork()` and `exec()`: create a child process to execute `command`.
- Return a file stream pointer that can be used with `stdio` library functions.

`popen()` creates a pipe + child process so it must be closed with `pclose()`.

Example: `popen()` a `ls` command

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    char buffer[254];

    FILE *fp = popen("ls -l", "r"); /* read mode */

    if(fp == NULL)
    {
        perror("popen");
        return -1;
    }

    /* Read the output line by line*/
    while(fgets(buffer, sizeof(buffer), fp) != NULL)
    {
        printf("%s", buffer);
    }

    pclose(fp);

    return 0;
}
```

```
total 64
-rw-r--r-- 1 dungvd dungvd   976 Sep 14 03:57 lab_1.c
-rwxr-xr-x 1 dungvd dungvd 16256 Sep 14 04:07 pipe
```

```
-rw-r--r-- 1 dungvd dungvd   950 Sep 14 04:33 pipe.c
-rwxr-xr-x 1 dungvd dungvd 16000 Sep 14 04:48 pipe2
-rw-r--r-- 1 dungvd dungvd   226 Sep 14 06:18 pipe2.c
-rwxr-xr-x 1 dungvd dungvd 16176 Sep 14 04:02 popen
-rw-r--r-- 1 dungvd dungvd   400 Sep 14 03:44 popen.c
```

`ls -l` writes its output into `stdout`, which is connected to a pipe. The parent process reads the output from the other end of the pipe.

2.4. FIFOs

FIFO is a named pipe.

FIFO has a name within the file system and is opened in the same way as a regular file, allowing communication between unrelated processes.

```
$ mkfifo [ -m mode ] pathname
```

```
#include <sys/stat.h>
int mkfifo(const char *pathname, mode_t mode);
/* Returns 0 on success, or –1 on error */
```

- `pathname`: the name of the FIFO to be created
- `mode`:
  file_perm.png

Example: FIFO tranfer between a server and a client

```
/* server.h */
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <signal.h>

void cleanup_fifo(int sig)
{
    unlink("./my_fifo");
    exit(0);
}
```

```
int main()
{
    char buffer[254];

    /* Create a FIFO */
    mkfifo("./my_fifo", S_IRUSR | S_IWUSR | S_IRGRP | S_IROTH);

    /* Clean up the FIFO on SIGINT */
    signal(SIGINT, cleanup_fifo);

    while (1)
    {
        /* Open the FIFO for reading */
        int fd = open("./my_fifo", O_RDONLY);
        if (fd == -1)
        {
            perror("open");
            return -1;
        }

        /* Read data from the FIFO */
        int bytes_read;
        while((bytes_read = read(fd, buffer, sizeof(buffer) - 1)) > 0)
        {
            buffer[bytes_read] = '\0';   // null-terminate the string
            printf("Received: %s\n", buffer);
        }

        /* Close the FIFO after sender is done */
        close(fd);
    }

    exit(0);
}
```

```
/* client.h */
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
```

```
#include <fcntl.h>
#include <sys/stat.h>

const char *FIFO_NAME = "./my_fifo";
const int NUMBER_OF_MESSAGES = 5;

int main()
{
    /* Open the FIFO for writing */
    int fd = open(FIFO_NAME, O_WRONLY);
    if (fd == -1)
    {
        perror("open");
        exit(-1);
    }

    /* Write messages to the FIFO */
    for(int i = 0; i < NUMBER_OF_MESSAGES; i++)
    {
        char message[50];
        snprintf(message, sizeof(message), "Message %d", i + 1);
        write(fd, message, sizeof(message));
        printf("Sent: %s\n", message);
        sleep(1);
    }

    close(fd);
    exit(0);
}
```

The server is responsible for creating the FIFO for communication. Therefore, we start the server first:

```
$ ./server
Server is up. Waiting for messages...
```

A FIFO appears in the file directory:

```
$ ls
client  client.c  my_fifo  server  server.c
```

Let's run the client, then the server can receive it through `my_fifo`:

```
./client
Sent: Message 1
Sent: Message 2
Sent: Message 3
Sent: Message 4
Sent: Message 5
```

Server:

```
Server is up. Waiting for messages...
Received: Message 1
Received: Message 2
Received: Message 3
Received: Message 4
Received: Message 5
```

---

3. Introduction to System V IPC

3.1. IPC Identifiers and Keys

IPC Identifier: An **internal integer name** assigned by the kernel to reference an active IPC object. It is a property of the object itself and is visible system-wide.

IPC Key: An **external naming scheme** of data type `key_t` used by cooperating processes to locate and agree on the same IPC object.

The **kernel** maintains data structures **mapping keys to identifiers** for each IPC mechanism.

Inspecting IPC objects:

```
$ ipcs        # list all IPC objects
$ ipcs -q     # message queues only
$ ipcs -s     # semaphores only
$ ipcs -m     # shared memory only
$ ipcrm       # remove an IPC object by id
```

Methods for generating IPC keys:

1. Randomly choose some integer key value, placed in a header file included by all programs using the IPC object (may accidentally choose a value used by another application)
2. Specifying `IPC_PRIVATE` guarantees that the kernel creates a new, unique IPC structure.

```
/* create a message queue*/
id = msgget(IPC_PRIVATE, S_IRUSR | S_IWUSR);
```

3. Generating Keys with `ftok()` (file to key):

```
#include <sys/ipc.h>
key_t ftok(char *pathname, int proj);
/* Returns integer key on success, or –1 on error */
```

3.2. Permission Structure

The associated data structure for an IPC object is initialized when the object is created via the appropriate `get` system call:

```
struct ipc_perm {
    key_t           __key;  /* Key, as supplied to 'get' call */
    uid_t           uid;    /* Owner's user ID */
    gid_t           gid;    /* Owner's group ID */
    uid_t           cuid;   /* Creator's user ID */
    gid_t           cgid;   /* Creator's group ID */
    unsigned short  mode;   /* Permissions */
    unsigned short  __seq;  /* Sequence number */
};
```

3.3. Configuration Limits

Message Queues

- **MSGMAX** (`/proc/sys/kernel/msgmax`): The maximum allowable size of a single message (in bytes)
- **MSGMNB** (`/proc/sys/kernel/msgmnb`): The default maximum capacity for a single message queue (in bytes)
- **MSGMNI** (`/proc/sys/kernel/msgmni`): The limit on the total number of message queues

Semaphores (`/proc/sys/kernel/sem`)

- **SEMMSL**: The maximum number of primitive semaphores per semaphore set.
- **SEMMNS**: The system-wide maximum number of semaphores across all sets.

- **SEMOPM**: The maximum number of operations permitted per `semop()` system call.
- **SEMMNI**: The system-wide maximum number of semaphore sets.

Shared Memory:

- **SHMMAX** (`/proc/sys/kernel/shmmax`): The maximum size of a single shared memory segment (in bytes)
- **SHMMNI** (`/proc/sys/kernel/shmmni`): The system-wide maximum number of shared memory segments
- **SHMALL** (`/proc/sys/kernel/shmall`): The total amount of shared memory, measured in pages, allocated system-wide.

Pipes & FIFOs:

- **PIPE_BUF**: The maximum number of bytes that can be written atomically to a pipe or FIFO without interleaving.
- **Pipe Capacity**: The maximum buffer capacity of a pipe (65,536 bytes since Linux 2.6.11), bounded per user by `/proc/sys/fs/pipe-max-size`

---

4. Message Queues

Message queues allow processes to exchange data in the form of messages.

msg_queue.png

Communication via message queues is message-oriented => the reader receives whole messages written by the writer.

It is **NOT possible** to:

- Read part of a message
- Leaving remainder in the queue
- Read multiple messages at a time

4.1. System V Message Queues

Interface	System V Message queues
Header file	`<sys/msg.h>`
Associated data structure	`msqid_ds`
Create/open object	`msgget()`

Interface	System V Message queues
Close object	(none)
Control operations	`msgctl()`
Performing IPC	`msgsnd()`-write message `msgrcv()`-read message

4.1.1. Creating or Opening a Message Queue

```
int msgget(key_t key, int msgflg);
/* Returns message queue identifier on success, or –1 on error */
```

- `msgflg`:
  - File permission bits.
  - `IPC_CREAT`: If no message queue with the specified key exists, create a new queue.
  - `IPC_EXCL`: If IPC_CREAT was also specified, and a queue already exists, fail with the error `EEXIST`

4.1.2. Sending Messages

```
int msgsnd(int msqid, const void *msgp, size_t msgsz, int msgflg);
/* Returns 0 on success, or –1 on error */
```

- `msgp`: Pointer to a programmer-defined structure:

  ```
  struct mymsg {
      long mtype;     /* Message type */
      char mtext[];   /* Message body */
  }
  ```

- `msgsz`: Size of `mtext` in bytes.
- `msgflg`:
  - `IPC_NOWAIT`: Perform a nonblocking send, and return `EAGAIN` when the queue is full.

A `msgsnd()` call that is blocked by a full queue may be interrupted by a signal handler. In this case, it fails with the error `EINTR`.

4.1.3. Receiving Messages

```
ssize_t msgrcv(int msqid, void *msgp, size_t maxmsgsz, long msgtyp, int msgflg);
/* Returns number of bytes copied into mtext field, or –1 on error */
```

`msgrcv()`: reads and removes a message from a message queue, and copies its contents into the buffer pointed to by msgp

- `maxmsgsz`: Maximum space of the buffer
  If `mtext` size > `maxmsgsz`, no message is removed and fails with the error `E2BIG`.
- `msgtyp`:
  - `== 0`: Take the first msg from the queue.
  - `> 0`: Take the first msg whose `mtype == msgtyp`.
  - `< 0`: Take the first msg with the smallest `mtype` such that `mtype <= abs(msgtyp)`.
- `msgflg`:
  - `IPC_NOWAIT`: Perform a nonblocking receive, and return `ENOMSG` when the queue is empty.
  - `MSG_EXCEPT`: If `msgtyp > 0`, Take the first msg whose `mtype != msgtyp`
  - `MSG_NOERROR`: If `mtext` size > `maxmsgsz`, take the msg by truncating the msg.

4.1.4. Message Queue Control Operations

```
int msgctl(int msqid, int cmd, struct msqid_ds *buf);
/* Returns 0 on success, or –1 on error */
```

- `IPC_RMID`: Immediately remove the message queue object, ignore the third argument.
- `IPC_STAT`: Get a copy of `msqid_ds` associated with this msg queue.
- `IPC_SET`: Update selected fields of `msqid_ds` associated with this msg queue.

```
struct msqid_ds {
    struct ipc_perm msg_perm; /* Ownership and permissions */
    time_t msg_stime; /* Time of last msgsnd() */
    time_t msg_rtime; /* Time of last msgrcv() */
    time_t msg_ctime; /* Time of last change */
    unsigned long __msg_cbytes; /* Number of bytes in queue */
    msgqnum_t msg_qnum; /* Number of messages in queue */
    msglen_t msg_qbytes; /* Maximum bytes in queue */
    pid_t msg_lspid; /* PID of last msgsnd() */
    pid_t msg_lrpid; /* PID of last msgrcv() */
};
```

Example: Sending a `long` value through a message queue

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/msg.h>
#include <string.h>
```

```
typedef struct MyMsg {
    long mtype;
    char mtext[1024];
} MyMsg;

int main()
{
    int msgid = msgget(123456, IPC_CREAT | 0666);

    if(msgid == -1)
    {
        perror("msgget");
        return -1;
    }

    switch (fork())
    {
        case -1:
            perror("fork");
            return -1;

        case 0: /* Child process recv */
            const int BUFFER_SIZE = 1024;
            MyMsg recv_msg;

            /* Receive a message from the message queue */
            msgrcv(msgid, &recv_msg, BUFFER_SIZE, 0, 0);

            /* Extract the long value from the message text */
            long recv_key;
            memcpy(&recv_key, recv_msg.mtext, sizeof(long));

            printf("Received msg: %li \n", recv_key);

            break;

        default: /* Parent process send */
            /* Initialize a message*/
            MyMsg send_msg;
            send_msg.mtype = 1;
```

```
            /* Send a long value by copying it to the message text */
            long super_important_key = 1234567890123456789;
            memcpy(send_msg.mtext, &super_important_key, sizeof(long));

            /* Send message to message queue*/
            msgsnd(msgid, &send_msg, sizeof(send_msg.mtext), 0);

            break;
    }

    return 0;
}
```

Output:

```
Received msg: 1234567890123456789
```

4.2. POSIX Message Queues

Interface	POSIX Message Queues
Header file	`<mqueue.h>`
Object handle	`mqd_t`
Create/open	`mq_open()`
Close	`mq_close()`
Unlink	`mq_unlink()`
Perform IPC	`mq_send()`, `mq_receive()`
Miscellaneous operations	`mq_setattr()` - set attributes `mq_getattr()` - get attributes `mq_notify()` - request notification

4.2.1. Opening, Closing, and Unlinking a Message Queue

```
mqd_t mq_open(const char *name, int oflag, ...
            /* mode_t mode, struct mq_attr *attr */);
/* Returns a message queue descriptor on success, or (mqd_t) –1 on error */
```

- `name`: name of the msg queue
- `oflag`:

    - `O_CREAT`: Create queue if it doesn't already exist
    - `O_EXCL`: With `O_CREAT`, create queue exclusively
    - `O_RDONLY`: Open for reading only
    - `O_WRONLY`: Open for writing only
    - `O_RDWR`: Open for reading and writing
    - `O_NONBLOCK`: Open in nonblocking mode
- `mode`: permission bits
- `attr`:

```
struct mq_attr {
long mq_flags; /* Message queue description flags: 0 or
O_NONBLOCK [mq_getattr(), mq_setattr()] */
long mq_maxmsg; /* Maximum number of messages on queue
[mq_open(), mq_getattr()] */
long mq_msgsize; /* Maximum message size (in bytes)
[mq_open(), mq_getattr()] */
long mq_curmsgs; /* Number of messages currently in queue
[mq_getattr()] */
};
```

```
int mq_close(mqd_t mqdes);
int mq_unlink(const char *name);
/* Returns 0 on success, or –1 on error */
```

4.2.2. Exchanging Messages

```
int mq_send(mqd_t mqdes, const char *msg_ptr, size_t msg_len, unsigned int msg_prio);
/* Returns 0 on success, or –1 on error */
```

```
ssize_t mq_receive(mqd_t mqdes, char *msg_ptr, size_t msg_len, unsigned int *msg_prio);
/* Returns number of bytes in received message on success, or –1 on error */
```

- `msg_len`: the length of the message pointed to by `msg_ptr`. Zero length are permitted
- `msg_prio`: nonnegative integer priority for a message.
    - Messages are ordered within the queue in descending order of priority (0 is the lowest priority).
    - When a new message is added to the queue, it is placed after any other messages of the same priority.

---

5. Semaphores

5.1. System V Semaphores

Interface	System V Semaphores
Header file	`<sys/sem.h>`
Associated data structure	`semid_ds`
Create/open object	`semget()`
Close object	(none)
Control operations	`semctl()`
Performing IPC	`semop()`-test/adjust semaphore

5.1.1. Creating or Opening a Semaphore Set

```
int semget(key_t key, int nsems, int semflg);
/* Returns semaphore set identifier on success, or –1 on error */
```

- `nsems`: number of semaphores in that set, must be > 0
  - For an existing set, `nsems` ≤ size of the existing set
- `semflg`:
  - Permission bits
  - `IPC_CREAT`, `IPC_EXCL`

The semaphores within a set are numbered starting at 0.

5.1.2. Semaphore Control Operations

```
int semctl(int semid, int semnum, int cmd, ... /* union semun arg */);
/* Returns nonnegative integer on success (see text); returns –1 on error */
```

- `semnum`: identifies a particular semaphore within the set.
- `arg`: a `semun` union

```
/* Definition of the semun union */


#ifndef SEMUN_H
#define SEMUN_H         /* Prevent accidental double inclusion */
```

```
#include <sys/types.h>  /* For portability */
#include <sys/sem.h>

union semun {           /* Used in calls to semctl() */
    int val;
    struct semid_ds * buf;
    unsigned short * array;
#if defined(__linux__)
    struct seminfo * __buf;
#endif
};

#endif
```

- `cmd`: the operation to be performed
  Generic control operations:
  - `IPC_RMID`: Immediately remove the semaphore set an `semid_ds` data structure. Any processes blocked are immediately awakened, with `semop()` reporting the error `EIDRM`.
  - `IPC_STAT`: Get a copy of the `semid_ds` to `arg.buf`.
  - `IPC_SET`: Update selected fields of the `semid_ds` in `arg.buf`.

  Retrieving and initializing semaphore values:
  - `GETVAL`:
    Returns the value of the `semnum`-th semaphore in the `semid` set.
  - `SETVAL`:
    The value of the `semnum`-th semaphore in the `semid` set is initialized to the value specified in `arg.val`.
  - `GETALL`:
    Retrieve the values of all of the semaphores in the `semid` set, placing them in the array pointed to by `arg.array`.
  - `SETALL`:
    Initialize all semaphores in the `semid` set, using the values supplied in the array pointed to by `arg.array`.

  Retrieving per-semaphore information:
  - `GETPID`:
    Return the PID of the last process to perform a `semop()` on this semaphore. If no process has yet performed a `semop()` on this semaphore, 0 is returned.
  - `GETNCNT`:
    Return the number of processes currently waiting for the value of this semaphore to increase.
  - `GETZCNT`:
    Return the number of processes currently waiting for the value of this semaphore to become 0.

```
struct semid_ds {
    struct ipc_perm sem_perm;   /* Ownership and permissions */
    time_t sem_otime;           /* Time of last semop() */
    time_t sem_ctime;           /* Time of last change */
    unsigned long sem_nsems;    /* Number of semaphores in set */
};
```

5.1.3. Semaphore Operations

`semop()` performs one or more operations on the semaphores in the semaphore `semid` set.

```
int semop(int semid, struct sembuf *sops, unsigned int nsops);
/* Returns 0 on success, or –1 on error */
```

- `sops`: an pointer to an array contains the operations to be performed

  ```
  struct sembuf {
      unsigned short sem_num; /* Semaphore number */
      short sem_op; /* Operation to be performed */
      short sem_flg; /* Operation flags (IPC_NOWAIT and SEM_UNDO) */
  };
  ```

  - `sem_num`: which one in the set this `sem_op` applies to
  - `sem_op`:
    - `> 0`: Add value to semaphore. May wake up other blocked processes. Needs alter (write) permission.
    - `== 0`: "Wait until zero" - checks if semaphore is 0 right now; if not, blocks until it is
    - `< 0`: Try to subtract `abs(sem_op)`. If enough value is available, do it immediately. Otherwise block until it is. Needs (write) alter permission.
- `nsops`: size of the array `sops` pointed to
- `sem_flg`:
  - `IPC_NOWAIT`: Non-blocking. Return `EAGAIN` instead of blocking.
  - `SEM_UNDO`: Undoes the operation if the process terminates

When a blocked `semop()` call unblocks?

1. Another process changed the semaphore value enough to satisfy the operation.
2. A signal interrupted it => fails with `EINTR` (`semop` is never auto-restarted after a signal)
3. Another process deleted the semaphore set => fails with `EIDRM`

5.2. POSIX Semaphores

There are 2 types of POSIX Semaphores:

- Named semaphore:
  - Has a name
  - Calling `sem_open()` with the same name, unrelated processes can access the same semaphore.
- Unnamed semaphore:
  - Doesn't have a name
  - Can be shared between processes or between a group of threads
  - When shared between processes, must reside in a region of shared memory (System V, POSIX, `mmap()`)
  - When shared between threads, reside in an area of memory shared by the threads (heap/global variable)

Interface	POSIX Semaphores
Header file	`<semaphore.h>`
Object handle	`sem_t *`
Create/open	`sem_open()`
Close	`sem_close()`
Unlink	`sem_unlink()`
Perform IPC	`sem_post()`, `sem_wait()`, `sem_getvalue()`
Miscellaneous operations	`sem_init()` - initialize unnamed semaphore `sem_destroy()` - destroy unnamed semaphore

5.2.1. Named Semaphore

Opening a Named Semaphore: creates and opens a new named semaphore or opens an existing semaphore

```
#include <fcntl.h> /* Defines O_* constants */
#include <sys/stat.h> /* Defines mode constants */
#include <semaphore.h>

sem_t *sem_open(const char *name, int oflag, ...
/* mode_t mode, unsigned int value */ );

/* Returns pointer to semaphore on success, or SEM_FAILED on error */
```

Closing a Semaphore: terminates the association of the semaphore

- Releases any resources of the semaphore for this process
- Decreases the count of processes referencing the semaphore

- Closing a semaphore does not delete it

```
int sem_close(sem_t *sem);


/* Returns 0 on success, or –1 on error */
```

Removing a Named Semaphore: removes the semaphore identified by `name` and marks the semaphore to be destroyed once all processes cease using it

```
int sem_unlink(const char *name);


/* Returns 0 on success, or –1 on error */
```

5.2.2. Unnamed Semaphore

Initializing an Unnamed Semaphore:

```
int sem_init(sem_t *sem, int pshared, unsigned int value);


/* Returns 0 on success, or –1 on error */
```

- `pshared`:
  - 0: shared between threads of the calling process
  - nonzero: shared between processes
- `value`: initial value of the semaphore

Destroying an Unnamed Semaphore:

```
int sem_destroy(sem_t *sem);


/* Returns 0 on success, or –1 on error */
```

- It is safe to destroy a semaphore only if no processes or threads are waiting on it.

5.2.3. Semaphore Operations

Waiting on a Semaphore: Decreases the value of the semaphore `sem` by 1

```
int sem_wait(sem_t *sem);
int sem_trywait(sem_t *sem);
int sem_timedwait(sem_t *sem, const struct timespec *abs_timeout);
```

```
/* Returns 0 on success, or –1 on error */
```

Posting a Semaphore: Increases the value of the semaphore `sem` by 1

```
int sem_post(sem_t *sem);


/* Returns 0 on success, or –1 on error */
```

Retrieving the Value of a Semaphore: returns the current value of the semaphore `sem` in the `int` pointed to by `sval`.

```
int sem_getvalue(sem_t *sem, int *sval);


/* Returns 0 on success, or –1 on error */
```

---

6. Shared Memory

Shared memory allows ≥ 2 processes to share the same region of physical memory.

This is the fastest form of IPC, because the data doesn't need to be copied between processes.

A shared memory segment is mapped to be part of a process's user-space memory, no kernel intervention is required for IPC.

shared_mem.png

Processes should implement a **synchronization technique** to access a shared memory.

6.1. System V Shared Memory

Interface	System V Shared Memory
Header file	`<sys/shm.h>`
Associated data structure	`msqid_ds`
Create/open object	`shmget()` + `shmat()`
Close object	`shmdt()`
Control operations	`shmctl()`
Performing IPC	access memory in shared region

6.1.1. Creating, Opening a Shared Memory Segment

`shmget()` creates a new shared memory segment or opening an existing segment

```
int shmget(key_t key, size_t size, int shmflg);
/* Returns shared memory segment identifier on success, or –1 on error */
```

- `size`: the desired size of the segment, in bytes (positive int)
  The kernel allocates shared memory in **multiples of the system page size**.
- `shmflg`:
  - permission bits
  - `IPC_CREAT`, `IPC_EXCL`
  - `SHM_HUGETLB` (since Linux 2.6): create a shared memory that uses *huge pages*.
  - `SHM_NORESERVE` (since Linux 2.6.15)

6.1.2. Attaching, Detaching a Shared Memory Segment

`shmat()` attaches the shared memory segment to the calling's virtual address space.

```
void *shmat(int shmid, const void *shmaddr, int shmflg);
/* Returns address at which shared memory is attached on success, or (void *) –1 on error */
```

- `shmaddr`:
  - `NULL`: Attach segment at a suitable address selected by the kernel (preferred method).
  - not `NULL`: Attach segment at the address specified by `shmaddr`, which must be a multiple of the system page size (or the error `EINVAL` results).
  - not `NULL`, `SHM_RND` is set: Attach segment at the address specified by `shmaddr`, round `shmaddr` down to multiple of `SHMLBA`.
- `shmflg`:
  - `0`: Default
  - `SHM_RDONLY`: Attach segment read-only
  - `SHM_REMAP`: Replace any existing mapping at `shmaddr`
  - `SHM_RND`: Round `shmaddr` down to multiple of `SHMLBA` (use with `shmaddr != NULL`)

`shmdt()`: detact the segment from its virtual address space. NOT deleting the shared memory.

```
int shmdt(const void *shmaddr);
/* Returns 0 on success, or –1 on error */
```

- `shmaddr`: the address returned by `shmat()`

6.1.3. Shared Memory Control Operations

```
int shmctl(int shmid, int cmd, struct shmid_ds *buf);
/* Returns 0 on success, or –1 on error */
```

Generic operations:

- `IPC_RMID`: Marks the segment for deletion.
  - If no process has the segment attached, immediately remove; otherwise removal happens once after all processes have detached from it.
- `IPC_STAT`: Copies the segment's `shmid_ds` structure into `buf`.
- `IPC_SET`: Updates selected fields of the segment's `shmid_ds` structure from `buf`.

Locking operations:

- `SHM_LOCK`: Locks the segment into RAM so it can't be swapped out, avoiding page-fault delays once pages are resident.
- `SHM_UNLOCK`: Unlocks it, allowing swapping again.

```
struct shmid_ds {
    struct ipc_perm shm_perm; /* Ownership and permissions */
    size_t shm_segsz; /* Size of segment in bytes */
    time_t shm_atime; /* Time of last shmat() */
    time_t shm_dtime; /* Time of last shmdt() */
    time_t shm_ctime; /* Time of last change */
    pid_t shm_cpid; /* PID of creator */
    pid_t shm_lpid; /* PID of last shmat() / shmdt() */
    shmatt_t shm_nattch; /* Number of currently attached processes */
};
```

Example: Shared memory is allocated in multiples of the system page size

Let's try creating a shared memory with only 10 bytes

```
#include <stdio.h>
#include <sys/shm.h>
#include <unistd.h>

int main()
{
    int shm_size = 10;

    /* Create a new shared memory segment */
```

```
    int shmid = shmget(IPC_PRIVATE, shm_size, IPC_CREAT | 0666);
    if (shmid == -1)
    {
        perror("shmget");
        return -1;
    }

    /* Attach the segment to virtual memory */
    void *baseaddr = shmat(shmid, NULL, 0);
    if (baseaddr == (void *)-1)
    {
        perror("shmat");
        return -1;
    }

    /* Mark for removal when no processes attached */
    shmctl(shmid, IPC_RMID, NULL);

    printf("Shared memory attached at: %p\n", baseaddr);
    printf("Process ID: %d\n", getpid());

    getchar();
    shmdt(baseaddr); /* Detach the segment */
    return 0;
}
```

Output:

```
Shared memory attached at: 0x706c1ed53000
Process ID: 41006
```

Inspection:

```
$ ipcs -m

------ Shared Memory Segments --------
key        shmid      owner      perms      bytes      nattch     status
0x00000000 14         dungvd     666        10         1
```

`ipcs` shows that we have a 10-byte shared memory

```
$ pmap -x 41006
Address           Kbytes     RSS   Dirty Mode  Mapping
...
0000706c1ed53000       4       0       0 rw-s-   [ shmid=0xe ]
...
```

But when we use `pmap` to inspect the memory map, the shared memory is 4kB = 4096 bytes large, which is a multiple of system page size:

```
$ getconf PAGESIZE
4096
```

6.2. Memory Mapping

6.2.1. Creating a Mapping: mmap()

`mmap()` creates a new memory mapping in the calling process's virtual address space

```
#include <sys/mman.h>
void *mmap(void *addr, size_t length, int prot, int flags, int fd, off_t offset);
/* Returns starting address of mapping on success, or MAP_FAILED on error */
```

- `addr`:
  - `== NULL`: let kernel pick the address
- `length`: size of the mapping in bytes
  - the kernel creates mappings by rounding up `length` in multiples of the page size
- `prot`: protection bit masks of the mapping
  - `PROT_NONE`: The region may not be accessed
  - `PROT_READ`: The contents of the region can be read
  - `PROT_WRITE`: The contents of the region can be modified
  - `PROT_EXEC`: The contents of the region can be executed
- `flags`:
  - `MAP_PRIVATE`: Modifications to the contents of the region are not visible to other processes employing the same mapping
  - `MAP_SHARED`: Modifications to the contents of the region are visible to other processes mapping the same region with the `MAP_SHARED` attribute

6.2.2. Unmapping a Mapped Region: munmap()

```
int munmap(void *addr, size_t length);
/* Returns 0 on success, or –1 on error */
```

- `addr`: the starting address of the address range to be unmapped, must be aligned to a page boundary.
- `length`: bytes of the region to be unmapped, the address range up to the next multiple of the system page size will be unmapped.

6.3. POSIX Shared Memory

Interface	POSIX Shared Memory
Header file	`<sys/mman.h>`
Object handle	`int (file descriptor)`
Create/open	`shm_open() + mmap()`
Close	`munmap()`
Unlink	`shm_unlink()`
Perform IPC	operate on locations in shared region

6.3.1. Creating Shared Memory Objects

```
int shm_open(const char *name, int oflag, mode_t mode);
/* Returns file descriptor on success, or –1 on error */
```

- `name`: name of the shared memory
- `oflag`:
  - `O_CREAT`: Create object if it doesn't already exist
  - `O_EXCL`: With `O_CREAT`, create object exclusively
  - `O_RDONLY`: Open for read-only access
  - `O_RDWR`: Open for read-write access
  - `O_TRUNC`: Truncate object to zero length
- `mode`: permission bits

6.3.2. Removing Shared Memory Objects

```
int shm_unlink(const char *name);
/* Returns 0 on success, or –1 on error */
```

- Doesn't affect existing mappings of the object.
- Prevents further `shm_open()` calls from opening the object.
- Once all processes have unmapped the object, the object is removed.

6.3.3. Using Shared Memory Objects

```
#include <sys/mman.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>

/* 1. Create/open the shared memory object */
int fd = shm_open("/my_shm", O_CREAT | O_RDWR, 0666);

/* 2. Size it */
ftruncate(fd, 4096);

/* 3. Map it into the address space */
void *addr = mmap(NULL, 4096, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);

/* 4. Close the fd */
close(fd);

/* 5. ... use addr like a normal pointer ... */

/* 6. Unmap and remove the shared memory */
munmap(addr, 4096);
shm_unlink("/my_shm");   // removes the name; object dies once last mapping goes away
```

6.4. Storing Pointers in Shared Memory

Each process may employ different shared libraries and memory mappings, and may attach different sets of shared memory segments.

It is unsafe to storing absolute pointers inside shared memory segments.

shm_ptr.png

Pointer offsets to base address return from `shmat()` should be used instead.

Example: Reproduce an error due to using storing an absolute pointer in a shared memory

```
/* writer */
#include <stdio.h>
#include <sys/shm.h>
```

```
int main()
{
    /* Create a new shared memory segment */
    int shmid = shmget(123, 4096, IPC_CREAT | 0666);
    if (shmid == -1)
    {
        perror("shmget");
        return -1;
    }

    /* Attach the segment to virtual memory */
    long *baseaddr = shmat(shmid, NULL, 0);
    if (baseaddr == (void *)-1)
    {
        perror("shmat");
        return -1;
    }

    /* Pointers for referencing the shared memory */
    long *p = &baseaddr[1];
    long *target = &baseaddr[50];

    /* Store a value in the shared memory */
    *target = 123456789;

    /* Store a pointer to target in the shared memory */
    *p = (long)target; /* Convert address to long*/

    /* Access the shared memory */
    printf("Memory addresses:\n");
    printf("- baseaddr of shared memory: %p\n", baseaddr);
    printf("- p in shared memory: %p\n", p);
    printf("- target in shared memory: %p\n", target);

    printf("Values:\n");
    printf("- target value: %ld\n", *target);
    printf("- p value: %p\n", (long *)*p);
    printf("- p dereferenced value: %ld\n", *(long *)(*p));
```

```
    /* Detach the segment */
    shmdt(baseaddr);
    return 0;
}
```

```
/* reader */
#include <stdio.h>
#include <sys/shm.h>

int main()
{
    /* Open an existed shared memory segment */
    int shmid = shmget(123, 4096, 0666);
    if (shmid == -1)
    {
        perror("shmget");
        return -1;
    }

    /* Attach the segment to virtual memory */
    long *baseaddr = shmat(shmid, NULL, 0);
    if (baseaddr == (void *)-1)
    {
        perror("shmat");
        return -1;
    }

    /* Mark for removal */
    shmctl(shmid, IPC_RMID, NULL);

    /* Pointers for referencing the shared memory */
    long *p = &baseaddr[1];
    long *target = &baseaddr[50];

    // *target = 123456789;
    // *p = (long)target;

    /* Access the shared memory */
    printf("Memory addresses:\n");
    printf("- baseaddr of shared memory: %p\n", baseaddr);
```

```
    printf("- p in shared memory: %p\n", p);
    printf("- target in shared memory: %p\n", target);

    printf("Values:\n");
    printf("- target value: %ld\n", *target);
    printf("- p value: %p\n", (long *)*p);
    printf("- p dereferenced value: %ld\n", *(long *)(*p));

    /* Detach the segment */
    shmdt(baseaddr);
    return 0;
}
```

Output:

```
$ ./writer
Memory addresses:
- baseaddr of shared memory: 0x7e74316fd000
- p in shared memory: 0x7e74316fd008
- target in shared memory: 0x7e74316fd190
Values:
- target value: 123456789
- p value: 0x7e74316fd190
- p dereferenced value: 123456789
```

```
$ ./reader
Memory addresses:
- baseaddr of shared memory: 0x7db4b7e11000
- p in shared memory: 0x7db4b7e11008
- target in shared memory: 0x7db4b7e11190
Values:
- target value: 123456789
- p value: 0x7e74316fd190
Segmentation fault         [ -x /usr/lib/command-not-found ]
```

As we can see that, the base addresses of the attached shared memory in these 2 processes' virtual memories are different, causing:

- `target` (stored in `baseaddr + 50`) is mapped to different addresses in 2 processes (`0x7e74316fd190` vs `0x7db4b7e11190`).

- the absolute pointer `p` (stored in `baseaddr + 1`, value `0x7e74316fd190`) is unable to work properly in the reader process, it points to an unallocated address. => Causing Segmentation fault

---

7. Signals Concept

7.1. What are signals?

Signals are software interrupts.

A signal is a notification to a process that an event has occurred.

Signal sources:

- The kernel
- Another process
- The process itself

Each signal is defined in `<signal.h>`:

- has a unique integer, starting from 1.
- has a name starting with `SIGxxxx`.

A signal is *generated* by some event, later then is *delivered* to a process.

7.2. Signals Dispositions

Signal default dispositions:

Each signal has a current disposition

- Term: terminate process
- Ign: ignore signal
- Core: terminate + dump core
- Stop: stop process
- Cont: resume a stopped process

Program disposition for a signal:

- Default action
- Ignore it

- Run a signal handler

7.3. Two types of signal

Traditional/Standard signals:

- Numbered 1 to 31.
- Each has a fixed, predefined meaning.
- Not queued:
  If multiple instances of a standard signal are generated while that signal is blocked, only one is marked is pending.
- No guaranteed delivery order.

Real-time signals:

- Numbered `SIGRTMIN` to `SIGRTMAX` (32-64 on Linux)
  `pthreads` lib uses 2-3 first real-time signals, therefore adjusts `SIGRTMIN` suitably to 34 or 35.
- Users define the meaning.
- Queued:
  If multiple instances of a real-time signal are generated while that signal is blocked, they are all queued as pending (limited by `RLIMIT_SIGPENDING`).
- Delivered in a guaranteed order (First In First Out).
- Can carry a small data payload (int or pointer) via `sigqueue()`.

8. Signal-related Functions

8.1. Changing Signal Dispositions: signal(), sigaction()

`SIGKILL` and `SIGSTOP` can't be dispositioned

8.1.1. signal()

```
void ( *signal(int sig, void (*handler)(int)) ) (int);
/* Returns previous signal disposition on success, or SIG_ERR on error */
```

- `sig`: signal number
- `handler`: a handler function (or `SIG_IGN` to ignore, `SIG_DFL` to reset to default)

```
void handler(int sig)
{
/* Code for the handler */
}
```

`signal()` behaves differently across Unix flavors => don't use it in real/portable code, use `sigaction()` instead

8.1.2. sigaction()

```
int sigaction(int sig, const struct sigaction *act, struct sigaction *oldact);
/* Returns 0 on success, or –1 on error */
```

```
struct sigaction {
 void (*sa_handler)(int);   /* Address of handler */
 sigset_t sa_mask;          /* Signals blocked during handler
 invocation */
 int sa_flags;              /* Flags controlling handler invocation */
 void (*sa_restorer)(void); /* Not for application use */
};
```

- `handler`: a handler function (or `SIG_IGN` to ignore, `SIG_DFL` to reset to default)
- `sa_mask`: signals to auto-block only during handler execution (avoids getting interrupted mid-handler).
- `sa_flags`:
  - `SA_RESTART`: auto-restart a system call that got interrupted by this handler
  - `SA_NODEFER`: don't auto-block this signal during its own handler (rare, riskier)
  - `SA_RESETHAND`: reset disposition back to default after one delivery (one-shot handler)
  - `SA_SIGINFO`: pass extra info (`siginfo_t`) to the handler
  - `SA_ONSTACK`: run handler on an alternate stack (sigaltstack())
  - `SA_NOCLDSTOP` / `SA_NOCLDWAIT`: tweak `SIGCHLD` behavior for stopped/dead children

8.2. Sending Signals: kill(), raise(), killpg()

```
int kill(pid_t pid, int sig);
/* Returns 0 on success, or –1 on error */
```

- `pid`:
  - `> 0`: send to that one process
  - `== 0`: send to every process in caller's own process group
  - `< -1`: send to every process in process group `abs(pid)`

- `== -1`: broadcast to every process caller has permission for (except PID 1 and the caller)

```
int raise(int sig);
/* Returns 0 on success, or nonzero on error */
```

`raise(sig)` is equivalent to `kill(getpid(), sig)`

```
int killpg(pid_t pgrp, int sig);
/* Returns 0 on success, or –1 on error */
```

`killpg(pgrp, sig)` is equivalent to `kill(-pgrp, sig)`

8.3. Signal Sets

```
int sigemptyset(sigset_t *set);
int sigfillset(sigset_t *set);
int sigaddset(sigset_t *set, int sig);
int sigdelset(sigset_t *set, int sig);
/* Both return 0 on success, or –1 on error */
```

One of `sigemptyset()` or `sigaddset()` must be used to initialize a signal set:

```
sigset_t set1;

sigemptyset(&set1);
sigaddset(&set1, SIGINT);
```

Now `set1` contains `SIGINT`

```
int sigismember(const sigset_t *set, int sig); /* test if sig is a member of set */
/* Returns 1 if sig is a member of set, otherwise 0 */

int sigandset(sigset_t *dest, sigset_t *left, sigset_t *right); /* dest = intersection of left
and right */
int sigorset(sigset_t *dest, sigset_t *left, sigset_t *right); /* dest = union of left and
right */
/* Both return 0 on success, or –1 on error */

int sigisemptyset(const sigset_t *set);
```

```
/* Returns 1 if sig is empty, otherwise 0 */
```

8.4. Signal Mask

A signal mask is a set of signals whose delivery to the process is currently blocked

```
int sigprocmask(int how, const sigset_t *set, sigset_t *oldset);
/* Returns 0 on success, or –1 on error */
```

- `how`:
  - `SIG_BLOCK`: add set to the mask
  - `SIG_UNBLOCK`: remove set from the mask
  - `SIG_SETMASK`: replace the mask outright with set
  - `NULL`: read the current mask into `oldset`
- `oldset`:
  - for saving the mask before changing

8.5. Pending Signals: sigpending()

If a process receives a signal that it is currently blocking, that signal is added to the process's set of pending signals

`sigpending()` gets the set of signals that are pending for the calling process.

```
int sigpending(sigset_t *set);
/* Returns 0 on success, or –1 on error */
```

8.6. Waiting for a Signal: pause()/sigsuspend()

8.6.1. pause()

`pause()` suspends the calling process until a signal handler interrupts

```
int pause(void);
/* Always returns –1 with errno set to EINTR */
```

Problem: Incorrectly unblocking and waiting for a signal

```
sigset_t intMask;
sigset_t emptyMask;

struct sigaction sa;

/* Initialize masks */
sigemptyset(&emptyMask);
sigemptyset(&intMask);
sigaddset(&intMask, SIGINT);

/* SIGINT disposition */
sigemptyset(&sa.sa_mask);
sa.sa_flags = 0;
sa.sa_handler = handler;
sigaction(SIGINT, &sa, NULL);

/* Block SIGINT */
sigprocmask(SIG_BLOCK, &intMask, NULL);

/* Critical section: Do some work here */

/* Unblock SIGINT */
sigprocmask(SIG_SETMASK, &emptyMask, NULL);

/* <-- BUG: what if SIGINT arrives here... */

pause();  /* Wait for SIGINT */
```

To avoid this problem, we require a means of atomically unblocking a signal and suspending the process. That is the purpose of the `sigsuspend()` system call.

8.6.2. sigsuspend()

```
int sigsuspend(const sigset_t *mask);
/* (Normally) returns –1 with errno set to EINTR */
```

`sigsuspend()` is equivalent to **atomically** performing these operations:

```
sigprocmask(SIG_SETMASK, &mask, &prevMask); /* Assign new mask */
pause();
sigprocmask(SIG_SETMASK, &prevMask, NULL); /* Restore old mask */
```

9. Lab

Link to lab

Lab 1

Implement a chat program between two processes over a pipe/FIFO

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <signal.h>
#include <pthread.h>
#include <string.h>

char user_name[50];
char friend_name[50];

char fifo_send[100];
char fifo_recv[100];
char recv_buffer[254];

pthread_t send_thread;
pthread_t recv_thread;

void* send_thread_func(void* arg)
{
    char send_buffer[254];
    while (1)
    {
        /* Read input from the user */
        if (fgets(send_buffer, sizeof(send_buffer), stdin) != NULL)
        {
            /* Open the FIFO for writing */
            int fd = open(fifo_send, O_WRONLY);
```

```
            if (fd == -1)
            {
                printf("%s is not online.\n", friend_name);
                continue;
            }

            /* Write data to the FIFO */
            write(fd, send_buffer, strlen(send_buffer));
            printf("You -> %s: %s", friend_name, send_buffer);

            /* Close the FIFO after writing */
            close(fd);
        }
    }
    return NULL;
}

void* recv_thread_func(void* arg)
{
    while (1)
    {
        /* Open the FIFO for reading */
        int fd = open(fifo_recv, O_RDONLY);
        if (fd == -1)
        {
            perror("open fifo_recv");
            printf("fifo_recv got deleted by accident?\n");
            exit(-1);
        }

        /* Read data from the FIFO */
        if(read(fd, recv_buffer, sizeof(recv_buffer) - 1) > 0)
        {
            printf("%s -> You: %s\n", friend_name, recv_buffer);
        }

        /* Close the FIFO after sender is done */
        close(fd);
    }
    return NULL;
```

```
}

void cleanup_fifo(int sig)
{
    unlink(fifo_recv);
    _exit(0);
}

int main(int argc, char *argv[])
{
    if (argc != 3)
    {
        fprintf(stderr, "Usage: %s <your_name> <friend_name>\n", argv[0]);
        return -1;
    }

    /* Store the user and friend names */
    snprintf(user_name, sizeof(user_name), "%s", argv[1]);
    snprintf(friend_name, sizeof(friend_name), "%s", argv[2]);

    /* Name the FIFOs based on names*/
    snprintf(fifo_send, sizeof(fifo_send), "%s_to_%s", argv[1], argv[2]);
    snprintf(fifo_recv, sizeof(fifo_recv), "%s_to_%s", argv[2], argv[1]);

    /* Create FIFOs */
    // mkfifo(fifo_send, 0666);
    mkfifo(fifo_recv, 0666); /* The client only create the receive FIFO */

    /* Clean up the FIFO on exit */
    signal(SIGINT, cleanup_fifo);
    signal(SIGKILL, cleanup_fifo);

    /* Create threads */
    pthread_create(&send_thread, NULL, send_thread_func, NULL);
    pthread_create(&recv_thread, NULL, recv_thread_func, NULL);

    pthread_join(send_thread, NULL);
    pthread_join(recv_thread, NULL);

    exit(0);
```

```
}
```

Result: Let's run the program in 2 terminals

```
# Tom's POV
$ ./lab_1 Tom Jerry
```

```
# Jerry's POV
$ ./lab_1 Jerry Tom
```

There will be 2 FIFOs appear in the current directory:

```
$ ls
Jerry_to_Tom  lab_1
Tom_to_Jerry  lab_1.c
```

Let's try to chat on both terminals:

```
# Tom's POV

Hey Jerry
You -> Jerry: Hey Jerry
Jerry -> You: Hi Tom

Can you go to my trap?
You -> Jerry: Can you go to my trap?
Jerry -> You: Hell nah
```

```
# Jerry's POV

Tom -> You: Hey Jerry
Hi Tom                                                  You -> Tom: Hi Tom
Tom -> You: Can you go to my trap?

Hell nah!
You -> Tom: Hell nah!
```

When a client ends the program:

```
# Jerry's POV
```

```
# Jerry does a Ctrl-C
```

```
# Tom's POV


...
Tom -> You: Can you go to my trap?

:Pleaseeeeee
Jerry is not online.
HEEEYYYYY
Jerry is not online
```

When that client opens the program again:

```
# Jerry's POV

$ ./lab_1 Jerry Tom
Hahahahaha
You -> Tom: Hahahahaha
Tom -> You: *sad*
```

```
# Tom's POV


...
HEEEYYYYY
Jerry is not online.
Jerry -> You: Hahahahaha

*sad*
You -> Jerry: *sad*
```

Lab 2

Implement producer-consumer via shared memory + semaphore

Summary:

- Producer: When empty, create until full, then wait for the consumption.
- Consumer: When full, consume until empty, then wait for the producer creation.

```
#include <stdio.h>
#include <unistd.h>
#include <mqueue.h>
#include <semaphore.h>
#include <sys/mman.h>
#include <signal.h>

struct shared_data {
    sem_t sem_full;
    sem_t sem_empty;
    int value;
};

const int MAX_SIZE = 5;
const char SHM_NAME[] = "/my_shm";
const size_t SHM_LENGTH = sizeof(struct shared_data);

int fd;     /* File descriptor for shared memory */
struct shared_data* addr;  /* Pointer to the shared memory */

void handler(int sig) {
    printf("Signal %d received. Cleaning up resource...\n", signum);

    /* Clean up semaphores */
    sem_destroy(&addr->sem_full);
    sem_destroy(&addr->sem_empty);

    /* Clean up shared memory */
    munmap(addr, SHM_LENGTH);
    shm_unlink(SHM_NAME);
    _exit(0);
}

void init_signal()
{
    struct sigaction sa;

    sigemptyset(&sa.sa_mask);
    sa.sa_handler = handler;
    sa.sa_flags = 0;
```

```
    sigaction(SIGINT, &sa, NULL);
}

void print_value(const char* name, const int value)
{
    printf("[%s] value = %i\n", name, value);
}

void producer(const char* name)
{
    /* Creation of shared memory */
    fd = shm_open(SHM_NAME, O_CREAT | O_RDWR, 0666);
    ftruncate(fd, SHM_LENGTH);
    addr = mmap(NULL, SHM_LENGTH, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
    if (addr == MAP_FAILED) {
        perror("mmap");
        return;
    }

    /* Initialize binary semaphores */
    sem_init(&addr->sem_empty, 1, 1);
    sem_init(&addr->sem_full, 1, 0);
    addr->value = 0;

    while(1)
    {
        sem_wait(&addr->sem_empty);

        /* Produce items */
        for(int i = 0; i < MAX_SIZE; i++)
        {
            addr->value++;
            print_value(name, addr->value);
            usleep(500000);
        }

        sem_post(&addr->sem_full);
    }
```

```
    /* Removal of shared memory */
    munmap(addr, SHM_LENGTH);
    shm_unlink(SHM_NAME);
}

void consumer(const char* name)
{
    /* Creation of shared memory */
    fd = shm_open(SHM_NAME, O_RDWR, 0666);
    ftruncate(fd, SHM_LENGTH);
    addr = mmap(NULL, SHM_LENGTH, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
    if (addr == MAP_FAILED) {
        perror("mmap");
        return;
    }

    while(1)
    {
        sem_wait(&addr->sem_full);

        /* Consume items */
        for(int i = 0; i < MAX_SIZE; i++)
        {
            addr->value--;
            print_value(name, addr->value);
            usleep(500000);
        }

        sem_post(&addr->sem_empty);
    }

    /* Removal of shared memory */
    munmap(addr, SHM_LENGTH);
    shm_unlink(SHM_NAME);
}

int main()
{
    init_signal();
```

```
    switch(fork())
    {
        case -1:
            perror("fork");
            return -1;
        case 0: // Child process
            usleep(500000); // Ensure producer starts first
            consumer("Consumer");
            break;
        default: // Parent process
            producer("Producer");
            break;
    }
    return 0;
}
```

Output:

```
[Producer] value = 1
[Producer] value = 2
[Producer] value = 3
[Producer] value = 4
[Producer] value = 5
[Consumer] value = 4
[Consumer] value = 3
[Consumer] value = 2
[Consumer] value = 1
[Consumer] value = 0
[Producer] value = 1
[Producer] value = 2
[Producer] value = 3
[Producer] value = 4
[Producer] value = 5
[Consumer] value = 4
[Consumer] value = 3
[Consumer] value = 2
[Consumer] value = 1
[Consumer] value = 0
...
```

Lab 3

Register a signal handler for SIGINT/SIGALRM; use strace to observe all the IPC-related syscalls

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void sigint_handler(int sig)
{
    printf("SIGINT received.\n");
    _exit(0);
}

void sigalrm_handler(int sig)
{
    printf("SIGALRM received.\n");
}

int main()
{
    struct sigaction sa;

    sa.sa_flags = 0;
    sigfillset(&sa.sa_mask);

    sa.sa_handler = sigint_handler;
    sigaction(SIGINT, &sa, NULL);
    sa.sa_handler = sigalrm_handler;
    sigaction(SIGALRM, &sa, NULL);

    printf("Set an alarm for 5 seconds.\n");
    alarm(5);

    printf("Waiting for signals...\n");
    pause();

    return 0;
}
```

Output:

Waiting for 5 seconds

```
./lab_3
Set an alarm for 5 seconds.
Waiting for signals...
SIGALRM received.
```

Sending a SIGINT (Ctrl-C)

```
$ ./lab_3
Set an alarm for 5 seconds.
Waiting for signals...
^CSIGINT received.
```

`strace` to observe all the IPC-related syscalls:

```
$ strace -e trace=ipc ./lab_3
Set an alarm for 5 seconds.
Waiting for signals...
--- SIGALRM {si_signo=SIGALRM, si_code=SI_KERNEL} ---
SIGALRM received.
+++ exited with 0 +++
```

Revision #15
Created 2026-09-15 03:43:43 UTC by Dung Vu
Updated 2026-09-21 07:23:04 UTC by Dung Vu